

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
INTERNATIONAL UNIVERSITY



PROJECT REPORT

COURSE: OBJECT-ORIENTED PROGRAMMING

Project: *Plants vs Zombies: Golden Spatula* (Greenfoot Framework)

Instructors: Ph.D Le Duy Tan, MSc.Nguyen Quang Phu

Team Leader: Le Anh Kiet

May 26, 2026

Team Members

No.	Full Name	Student ID	Role	Core Responsibilities
1	Le Anh Kiet	ITCSIU24047	Plant, Zombie, System, UI	Ensuring core game logic runs stably.
2	Le Thanh Hoang	ITCSIU24028	UI	Designing and programming the user interface.
3	Nguyen Quoc Trung Kien	ITCSIU24046	Zombie	Developing subclasses and special Zombies.
4	Duong Gia Thuc	ITCSIU24084	Zombie	Developing subclasses and special Zombies.
5	Nguyen Viet Huy	ITDSIU24021	None / Pending	No tasks completed during the project.

Contents

1	Project Introduction	4
2	Game Overview	5
2.1	Core Mechanics	5
2.2	Illustrations	5
3	System Architecture	7
3.1	Core Components in the System	7
3.2	Interaction Mechanisms and Operational Flow	7
4	Plant Class Architecture	8
4.1	Delegation Design Architecture	8
4.2	State Machine and Fusion Mechanics	8
4.3	Technical Specifications of Key Entities	9
4.4	Core Operational Mechanics	9
5	Zombie Class	11
5.1	Object-Oriented Architecture and State Machine	11
5.2	Classification and Entity Configuration	11
5.3	Combat and Collision Mechanics	11
5.4	Multi-stage Death Handling Pipeline	12
5.5	Audio Feedback Integration	12
6	Game Flow and State Lifecycle	13
6.1	Game Flow State Diagram	13
6.2	Detailed Operational Phases	14
7	Application of SOLID Principles	15
7.1	Single Responsibility Principle (SRP)	15
7.2	Open/Closed Principle (OCP)	15
7.3	Liskov Substitution Principle (LSP)	15
7.4	Interface Segregation Principle (ISP)	15
7.5	Dependency Inversion Principle (DIP)	16
8	Design Patterns	17
8.1	Factory Method Pattern (Entity Initialization)	17
8.2	Facade Pattern	19
8.3	Flyweight Pattern	20
8.4	Template Method Pattern	21
8.5	State Pattern	23
8.6	Strategy Pattern	26
8.7	Command Pattern	28
8.8	Observer Pattern (Event Bus)	29
9	Conclusion and Future Work	30
10	Links	31

1 Project Introduction

The project develops an innovative gameplay experience by combining the classic tower defense (*Tower Defense*) mechanics of *Plants vs. Zombies* with the tactical depth of the auto-battler (*Auto-battler* / *Teamfight Tactics*) genre. Breaking away from traditional linear defense models, the game introduces players to a dynamic arena centered around resource management and strategic team positioning.

- **Hexagonal Grid Arena:** Replaces the conventional lane-based system with a hexagonal grid layout, expanding multi-directional combat space and optimizing tactical formation deployment.
- **Card Rolling Mechanism (Roll Gacha):** Players spend *Sun* resources to refresh the shop (*Roll*), dynamically adapting to randomized unit pools to assemble powerful synergistic (*Synergy*) team compositions.
- **Star-Level Merging (Merge System):** Upon accumulating identical duplicates of an entity, the system automatically triggers an evolutionary mechanism, increasing the star level to simultaneously alter visual appearance and amplify combat statistics.



Figure 1: Gameplay Overview.

2 Game Overview

The game operates on a fixed grid system, where players must strategically deploy plant units to repel waves of advancing zombies. The core mechanics revolve around resource management and tactical defensive positioning.

2.1 Core Mechanics

- **Grid Placement System:** Plants are deployed into grid cells, with each entity occupying a single cell. Placing plants consumes "Sun" resources, which serve as the primary currency in the game.
- **Resource System (Sun System):** Sun resources drop randomly on the screen, and players must collect them to accumulate currency.
- **Waves and Lanes:** Zombies spawn in timed waves and advance along fixed horizontal lanes towards the player's left flank.
- **Projectiles and Collision:** Plants attack using projectiles (such as Pea, FirePea, Needle) or close-range melee attacks. Projectiles travel along the lane and deal damage upon colliding with zombies.
- **Termination Conditions (Game Over):** The game ends in defeat when a zombie breaches the final column of the grid after the last defensive item (*Lawn Mower*) in that specific lane has already been triggered and depleted.

2.2 Illustrations



Figure 2: Augment Card.



Figure 3: Each successfully defended wave rewards the player with 300 suns.

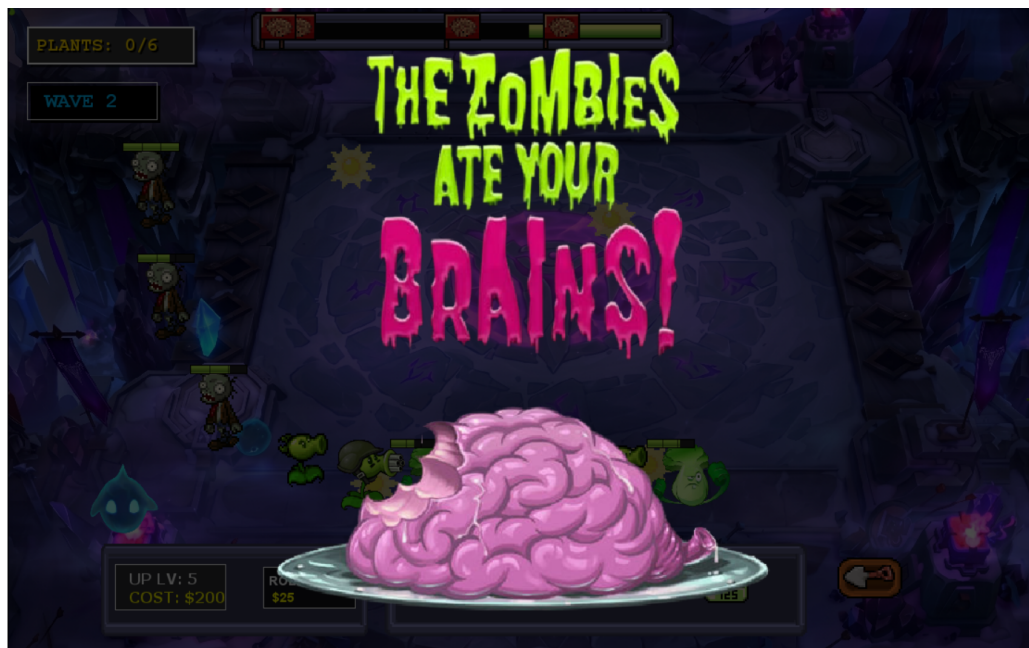


Figure 4: GameOver Screen.



Figure 5: GameOver Screen (with Retry option).

3 System Architecture

The two primary context classes inheriting from `Greenfoot.World` are `Arena` (coordinating cutscenes, dialogues, and background scrolling) and `PlayScene` (coordinating the active match). Both act as the central controllers (*Controller*), collaborating with specialized managers (*Managers*) to strictly enforce the Single Responsibility Principle (SRP).

3.1 Core Components in the System

3.2 Interaction Mechanisms and Operational Flow

To maintain loose coupling (*Loose Coupling*), complex programmatic tasks are decomposed into specialized handlers:

- **Automated Scene Transition:** `Arena` utilizes a finite state machine (*State Machine*) to run the background scrolling timeline (`runScrollSequence`), clean up active resources, and automatically initialize the `PlayScene`.
- **Instantiation Abstraction:** Entity spawning (Suns, Plants, Zombies) is completely delegated to respective Factories and Spawners (`SunSpawner`), isolating them from the `World` core logic.
- **State Encapsulation:** Victory and defeat evaluation structures are isolated inside the `WinLossHandler`, while peripheral system interactions (such as pressing the Escape key to call the `SettingsMenuPanel`) are processed through independent callback functions.

Component / Class	Core Responsibility in OOP Architecture
Arena & PlayScene	Manage global core states (<code>GameState</code> , <code>isPaused</code>). <code>Arena</code> handles camera scrolling (<code>applyScroll</code>) and dialogue text; <code>PlayScene</code> orchestrates the main combat loop.
WaveManager & ZombieFactory	<code>WaveManager</code> controls monster spawning waves driven by <code>LevelConfig</code> . Concrete <code>Zombie</code> subclasses extend a unified behavioral blueprint.
SeedBank & PlantFactory	<code>PlantFactory</code> applies the <i>Factory</i> pattern to generate unit cards (<code>SeedPacket</code>). <code>SeedBank</code> manages the player's active card inventory.
GridManager & Hitbox	Map asymmetric hexagonal layouts, validate structural placement rules (<code>canPlace</code>), and capture overlapping entity references via proximity collisions.
UpgradeManager & Merger	Handle the evolutionary merging mechanics (<i>Merge</i>) and sync functional side-effects via the <code>PlantEventBus</code> event broker.
AudioManager	Independently encapsulate sound execution logic (BGM/SFX) across the entire game lifecycle.

Table 1: Object management components within the system.

4 Plant Class Architecture

4.1 Delegation Design Architecture

The abstract `Plant` class uses a Delegation pattern to split responsibilities into specialized sub-components, preventing monolithic code bloating within the main execution loop:

- **PlantStateManager:** Controls the entity's active actions, managing state validation via `canAct()` and `isBusy()` hooks.
- **PlantInputHandler:** Encapsulates low-level mouse listening routines and coordinates user drag-and-drop mechanics.
- **PlantDeathHandler:** Executes plant expiration, playing `gulp.mp3` and clearing tracking records via `GridManager`.
- **PlantEventBus:** Publishes critical lifecycle signals (`publishStateChanged`, `publishHit`, `publishDeath`) directly to decoupling brokers.

4.2 State Machine and Fusion Mechanics

The entity's timeline is governed by a finite state machine executed directly inside `Greenfoot`'s synchronous `act()` ticks:

- **Visual Masking:** Operates via `updateTransparency()` to adjust alpha bounds dynamically (125 for dragging, 160 for preview, 255 for combat rendering).
- **Fusion Translation:** Triggered by `setMergingTarget()`. The entity unregisters from the board layout and moves toward its destination vector at a constant speed of 15.0 pixels per frame.

- **Atomic Fusion Finalization:** When two merging units collide (`notifyMergerArrived()` ≥ 2), it invokes `finalizeUpgrade()`. This transaction removes parent assets, queries `UpgradeManager.getUpgradeResult()`, spawns the tier-up entity onto the corresponding cell grid, and triggers a new `Dirt()` visual element.

4.3 Technical Specifications of Key Entities

Plant variants are classified based on combat statistics and specific attack mechanics defined within the `PlantType` enum:

Table 2: Detailed technical specifications of plant entities (Updated)

Entity Name	HP	Sun	Damage	Delay	Core Characteristic Mechanics
PEASHOOTER	200	100	35	1500ms	Single shot, <i>Normal</i> damage type.
REPEATER	200	125	50	1700ms	Burst shot (<i>Burst</i>) firing 2 consecutive peas.
GATLING PEA	200	300	50	800ms	Rapid fire, continuous 7-pea burst sequence.
GATLING PEA 2	200	400	50	2000ms	High firepower, heavy <i>Fire</i> damage type.
CACTUS (Lvl 1)	200	125+	30	1500ms	Piercing <i>Needle</i> damage type.
CACTUS (Lvl 2)	200	250	30	1200ms	Fires 2 projectiles simultaneously; piercing <i>Needle</i> damage type.
CACTUS (Lvl 3)	200	375	30	1000ms	Fires 3 projectiles simultaneously; piercing <i>Needle</i> damage type.
BONK CHOY	300	175	15	300ms	Melee strike, extremely fast attack speed.
BONK CHOY 2	600	400	25	200ms	Apex melee tier, massive HP and damage scale.
BONK CHOY 3	900	600	30	150ms	Apex melee tier, massive HP and damage scale.

4.4 Core Operational Mechanics

4.4.1 Evolutionary Upgrade Logic (Merge System)

When the entity merging process is triggered, the `Plant` class executes the following behavioral pipeline:

1. **Detach:** Removes its logical reference from the `GridManager` to immediately release its grid cell occupancy.
2. **Transition:** Interpolates linearly towards the target position via `handleMergeMovement()` at a velocity of 15.0 px/frame.
3. **Finalize:** Instantiates a `Merger` object to substitute the old entities with a higher-tier unit once the remaining target distance is ≤ 15 px.

4.4.2 Survival Management and Grid Synchronization

- **Lifecycle:** The `hit()` method processes incoming damage but will **ignore** it entirely if the entity is currently in a *Dragging* or *Merging* state. When HP reaches 0, the system fires a signal through the `EventBus`, frees the grid allocation, and spawns a `Dirt` particle effect before destroying the object instance.
- **Grid Sync:** The `syncGridPosition()` method performs translation mapping from Pixel coordinates to Logical coordinates using matrix data from the `GridManager`, ensuring that advancing Zombie entities can accurately identify their target lanes (*lane*).

5 Zombie Class

5.1 Object-Oriented Architecture and State Machine

The `Zombie` class is designed as an abstract class (*abstract class*), implementing the `IDamageable` and `IGridObject` interfaces. The system operates on a hybrid architecture combining the **State Pattern** for behavioral management and an **Event Bus** to coordinate decoupling communication between sub-systems:

- **IZombieState:** A unified interface for the *Walking*, *Eating*, and *Dying* states. Switching states via `setState()` encapsulates structural movement and combat execution logic.
- **ZombieEventBus:** Dispatches asynchronous signals regarding *Hit*, *Death*, and *StateChange* events to peripheral modules (UI, Audio) without triggering circular dependencies (*Circular Dependency*).

5.2 Classification and Entity Configuration

Attribute parameters are strictly encapsulated within `ZombieConfig`, allowing independent game balance adjustments decoupled from the core procedural source code.

Table 3: Technical specifications and role classification of `Zombie` entities

Zombie Type	HP	Speed	DMG	Classification & Technical Attributes
BASIC ZOMBIE	300	1.0	20	Standard: Core unit, defining the fundamental pacing of the level.
CONEHEAD	900	1.0	20	Light Tanker: Moderate durability, increasing layout pressure.
BUCKETHEAD	1000	1.0	30	Heavy Tanker: High defensive stats and threat scaling.
BRICKHEAD	1300	0.75	30	Super Tanker: Apex durability tier, slow translation speed.
PIANO ZOMBIE	2000	0.8	20	Mini-Boss: High health pool, demanding sustained DPS.
RA ZOMBIE	600	1.0	20	Disruptor: Interactive utility, siphoning active sun resources.
EXCAVATOR	1500	1.0	20	Counter-Unit: Hard counter to pure melee units (BonkChoy).
NUTCRACKER	700	1.0	20	Striker: High burst-damage capabilities in brief windows.
SUN ZOMBIE	250	1.5	0	Support/Loot: Rapid velocity, drops supportive loot upon termination.

5.3 Combat and Collision Mechanics

Zombies determine active targets using structural matrix lookups from the `GridManager` on their corresponding horizontal lane (*lane*):

- **Collision Detection:** Scans the specific `Board` row at `getYPos()`. A collision is registered when:

$$\Delta X = X_{\text{Zombie}} - X_{\text{Plant}} < 40 \text{ pixels}$$

- **Frame-Sync Animation:** Damage values are strictly applied at frames 2 or 5 of the eating animation sequence to match visual feedback. An `eatOnce` state flag ensures damage single-execution uniqueness per bite cycle.

5.4 Multi-stage Death Handling Pipeline

To replicate classic PvZ game characteristics, the termination lifecycle is divided into multi-stage logical sequences:

1. **Stage 1 (Decapitation):** Upon HP depletion, the entity ceases taking damage, triggers its death event, and instantiates a standalone `Head` object governed by a free-falling physics vector.
2. **Stage 2 (Collapse):** Instantiates a `FallingZombie` actor to execute the collapsing animation on the field, after which the original entity instance is safely purged from system memory (`removeObject`).

5.5 Audio Feedback Integration

The system routes signals to the `AudioManager` to play sound assets matching current runtime states:

- **Chomp:** Triggers *"chomp.mp3"* in frame-synchronization with active damage frames.
- **Death SFX:** Triggers *"limbs_pop.mp3"* during armor/head detachment and a *"zombie_falling.mp3"* sound sequence upon concluding Stage 2.

6 Game Flow and State Lifecycle

The system controls the game's lifecycle and operational flow through Greenfoot's core `act()` method, dividing it into 3 main logical phases combined between **Arena** and **PlayScene**.

6.1 Game Flow State Diagram

The state diagram below illustrates the transition sequence from resource loading, dialogues, through to the combat loop and level termination.

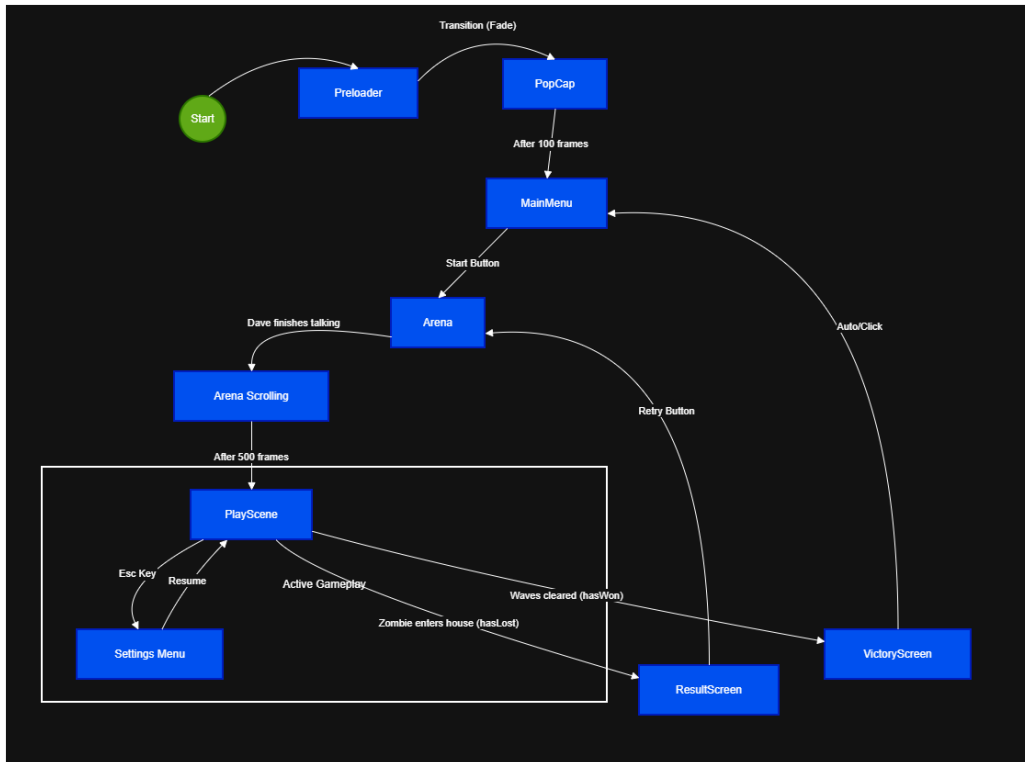


Figure 6: System State Transition and Gameplay Flow Diagram.

6.2 Detailed Operational Phases

Operational Phase	Detailed Core Logic and Mechanism
1. Initialization	• Triggers Crazy Dave's dialogue sequence (DAVE_TALKING). • Transitions the global state machine to SCROLLING. • Executes horizontal camera movement across the lawn (applyScroll) for 500 frames to preview upcoming enemies. • Spawns and transfers control to the PlayScene world environment (1111×698).
2. Main Gameplay Loop	• Interaction Handling: Monitors the "escape" key to toggle SettingsMenuPanel. Updates mouse-following Hitbox for real-time validation during tryPlacePlant(). Deducts 25 sun resources via RollButton to trigger random card refreshes based on rarity weights. • Logic Updates: WaveManager processes timeline wave data. SunSpawner handles periodic resource generation. updateMergers() continuously evaluates and executes multi-frame plant merging mechanics. • Rendering & Z-Index: Enforces applyDefaultPaintOrder() to structurally overlay UI components above game entities (Plant/Zombie). Invokes drawWaveUI() to render level progression metrics.
3. Termination Check	• Defeat Condition (<i>hasLost</i>): Triggered if any active Zombie instance breaches the final defensive parameter ($x < 155$). Instantly switches the context to ResultScreen, offering a <i>Retry</i> option back to the Arena. • Victory Condition (<i>hasWon</i>): Evaluated when WaveManager flags all waves completed and the entity registry contains zero remaining monsters. Routes the system to VictoryScreen before returning to the MainMenu.

7 Application of SOLID Principles

Applying **SOLID** principles serves as a guide to transition the system from traditional OOP thinking into a **Decoupled Architecture**, ensuring scalability as entity counts grow.

7.1 Single Responsibility Principle (SRP)

"A class should have only one reason to change."

The system isolates distinct responsibilities to avoid "God Classes":

- **Resources:** `SpriteCache` handles image caching, entirely decoupled from the animation logic of `SpriteAnimator`.
- **Initialization:** `PlantFactory` and `ZombieFactory` exclusively manage entity configuration and spawning logic.
- **Events:** `PlantEventBus` handles message broker duties, decoupling entity state changes from UI or Sound reactions.

7.2 Open/Closed Principle (OCP)

"Open for extension, closed for modification."

This allows seamless integration of new content without modifying the stable core codebase:

- **Entities:** The abstract class `Plant` defines the base framework. New plants (`BonkChoy`, `Cactus`) simply extend it without modifying the base source code.
- **States:** Utilizes `PlantState` and `IZombieState` interfaces to add new behaviors via polymorphism instead of bloating `switch-case` blocks.

7.3 Liskov Substitution Principle (LSP)

"Subtypes must be substitutable for their base types without breaking correctness."

Concrete entities (`Peashooter`, `BasicZombie`) seamlessly substitute their abstract parent classes. `GridManager` interacts uniformly with all `Plant` objects via common methods (`hit()`, `setGridPosition()`) without concrete casting, eliminating runtime logical errors during grid swapping.

7.4 Interface Segregation Principle (ISP)

"Many client-specific interfaces are better than one general-purpose interface."

Behaviors are broken down into granular interfaces, preventing subclasses from executing redundant methods:

- **IDamageable:** Manages health and damage reception mechanics.
- **IGridObject:** Defines entity positioning on the coordinate grid system.
- **IEatable:** Marks targets that can be consumed by zombies.

Example: Projectile entities implement only `IGridObject` for positioning, remaining entirely unaffected by `IDamageable` or `IEatable`.

7.5 Dependency Inversion Principle (DIP)

"Depend upon abstractions, not concretions."

This is the core solution to eliminate tight coupling across the system:

- **UpgradeManager** communicates via the **IPlantFactory** abstraction to spawn units and listens to merge states through event systems.
- **Listener Systems:** High-level modules (Audio, UI) implement **IPlantEventListener**. When an entity dies, it fires an event to the Bus. The Bus dispatches it to active listeners without the entity knowing these sub-systems exist.

8 Design Patterns

The project applies standard design patterns to optimize the management of complex entities, supporting the integration between tower defense gameplay and auto-battler mechanics.

8.1 Factory Method Pattern (Entity Initialization)

- **Application:** Isolates and abstracts the entire instantiation logic of Plants and Zombies away from the core game control flow and level orchestration.
- **Structural Evidence:**
 - **Plant System:** The `IPlantFactory` interface manages the production of defensive units. Its concrete implementation `PlantFactory` (utilizing the *Singleton* pattern) evaluates the `PlantType` enum to return abstract `IPlant` references.
 - **Zombie System:** Driven by the `IZombieFactory` interface and realized via the concrete `ZombieFactory` (Singleton). It overloads `createZombie()` to parse either `ZombieType` enums or map-defined `String` identifiers, instantiating variants (`BasicZombie`, `Brickhead`, etc.) as abstract `Zombie` products.
 - **Client Integration:** The `WaveManager` couples exclusively with the `IZombieFactory` interface inside its `spawnZombies()` routine, remaining completely decoupled from concrete enemy classes.
- **Benefits:** Enforces the **Open/Closed Principle (OCP)**. New entity variants can be seamlessly introduced by implementing the base product interfaces and appending an explicit enumeration branch inside the factories, leaving core scene and wave engines completely untouched.

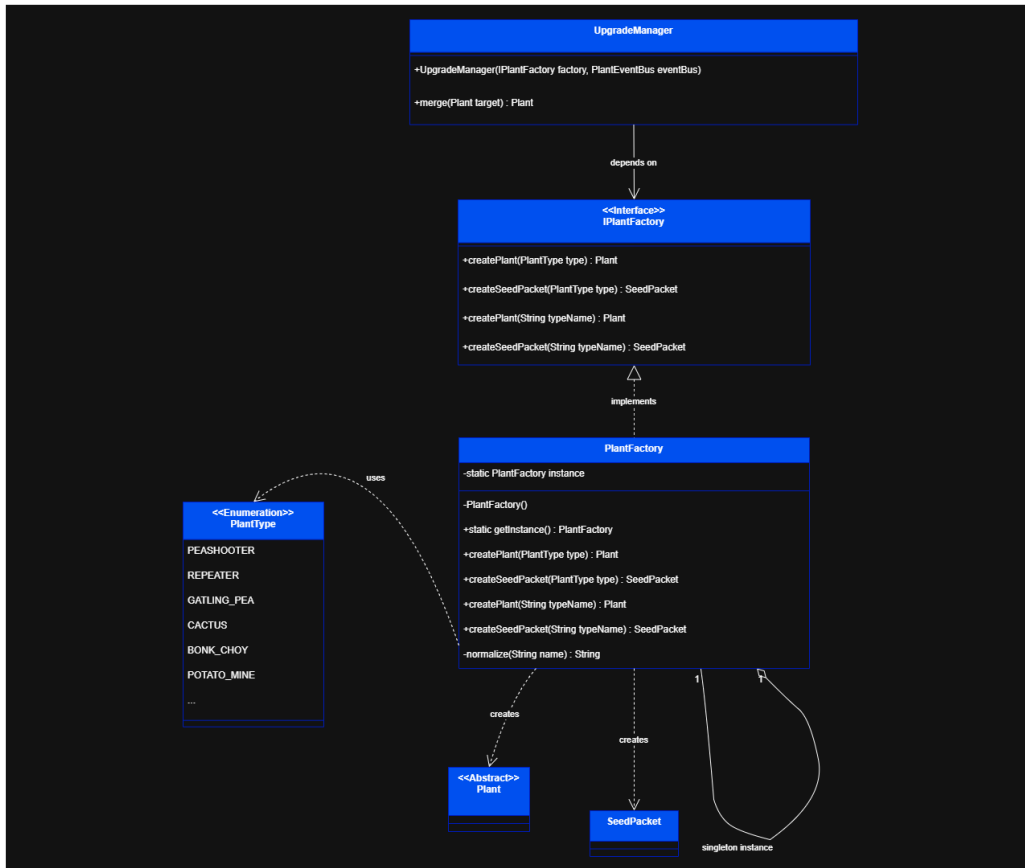


Figure 7: Factory pattern class diagram for Plant initialization.



Figure 8: Factory Method Pattern architecture decoupling entity generation from WaveManager control loop.

8.2 Facade Pattern

8.2.1 Design Intent

The Facade Pattern introduces `PlayScene` as a unified, high-level interface to encapsulate complex subsystem managers, mitigating tight coupling between the peripheral Greenfoot environment and internal game logic.

8.2.2 Implementation Components & Evidence

- **Facade (`PlayScene`):** Acts as the singular entry point for level-wide coordination.
- **Subsystems:** Specialized controllers including `SunManager`, `GridManager`, `MusicController`, `UpgradeManager`, and `WinLossHandler`.
- **Concrete Transaction:** When initiating plant placement, the client invokes a single method: `tryPlacePlant()`. The `PlayScene` facade autonomously coordinates the underlying pipeline: it validates costs via `SunManager.hasEnough()` and maps spatial nodes via `GridManager.placePlant()`, isolating subsystem complexities from the caller.

8.2.3 Benefits Achieved

- **Simplification & Loose Coupling:** Establishes structural boundaries. Changes to the internal data structures or algorithmic implementations within any downstream manager class leave client-side source code completely unaffected.

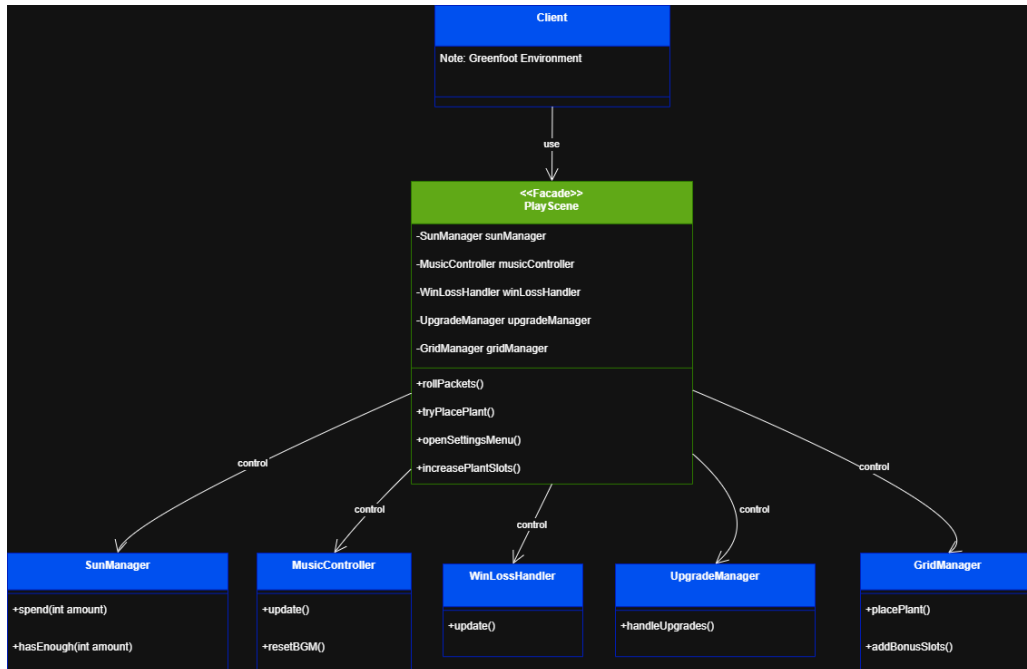


Figure 9: Facade Pattern model coordinating subsystems within `PlayScene`.

8.3 Flyweight Pattern

8.3.1 Design Intent

The Flyweight Pattern is applied through the `SpriteCache` class to share heavy graphical resources (`GreenfootImage`) across multiple entities (`Plant` or `Zombie`), optimizing RAM utilization and preventing redundant loading of animation frames when entity counts spike on the screen.

8.3.2 Implementation Components

The architecture isolates entity data into two distinct states:

- **Intrinsic State:** Shared, immutable data — specifically, the animation frame arrays centrally stored within `SpriteCache` using a `Map<String, GreenfootImage[]>` structure.
- **Extrinsic State:** Variant data that changes per unique entity instance (such as coordinates `x`, `y`, current health points `hp`, and active frame indices `frame`), which is maintained directly inside the respective concrete subclass of the entity.

8.3.3 Concrete Evidence

The animation controller class `SpriteAnimator` does not instantiate images or execute disk I/O queries by itself; instead, it requests graphical resources via `SpriteCache.importSprites()`. If the animation sequence of an entity type (e.g., `"Peashooter_Idle"`) already exists within the cache, `SpriteCache` immediately returns a reference to that existing array. Consequently, dozens of active `Peashooter` units on the lawn share a singular block of memory for their graphics data.

8.3.4 Benefits Achieved

- **Memory Efficiency:** Minimizes the memory footprint from a product of (N entities $\times M$ frames) down to a single set of images per entity type.
- **Performance Optimization:** Eliminates disk I/O latency when spawning new entities, while supporting centralized memory reclamation via the `clearCache()` method during scene transitions.

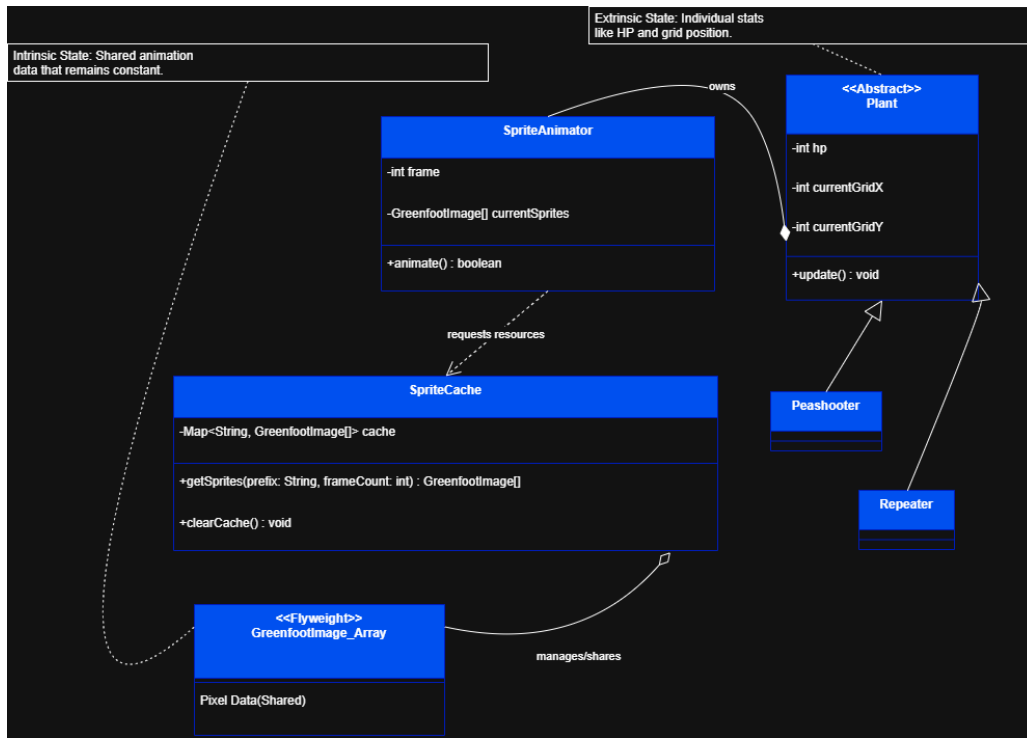


Figure 10: Flyweight Pattern model separating intrinsic and extrinsic states for resource management.

8.4 Template Method Pattern

8.4.1 Design Intent

The Template Method Pattern establishes an invariant lifecycle and reactive algorithm skeleton in abstract base classes (`Plant/Zombie`), enabling concrete subclasses to override specific behaviors without disrupting the execution sequence.

8.4.2 Implementation Components & Evidence

- **Template Method:** Superclass operational workflows (e.g., standard world registration via `addedToWorld()` and state/health verification loops).
- **Plant System** primitive `update()` is implemented by `BonkChoy` to trigger proximity-based melee logic (`handleCombat()`). It overrides the `hit(int dmg)` hook to inject contextual state-based visual flashes (`hitFlash()`) before delegating resource reduction to `super.hit(dmg)`.
- **Zombie System** primitives force subclasses like `BasicZombie` to implement `update()` for frame-by-frame orchestration and `getCurrentAnimation(boolean isEating)` to return conditional sprite arrays dependent on state (`isArmless/eating`). Overridden hooks (`handleThresholds()`, `deathAnim()`) dynamically instantiate debris objects (`new Arm()`, `new Head()`) upon health expiration.

8.4.3 Benefits Achieved

- **Code Reusability & Maintainability:** Centralizes boilerplate infrastructure (HP bars, reference caching, global combat tick rules) within base layers. Core rule

updates require a single refactoring entry point, minimizing code duplication across entity variants.

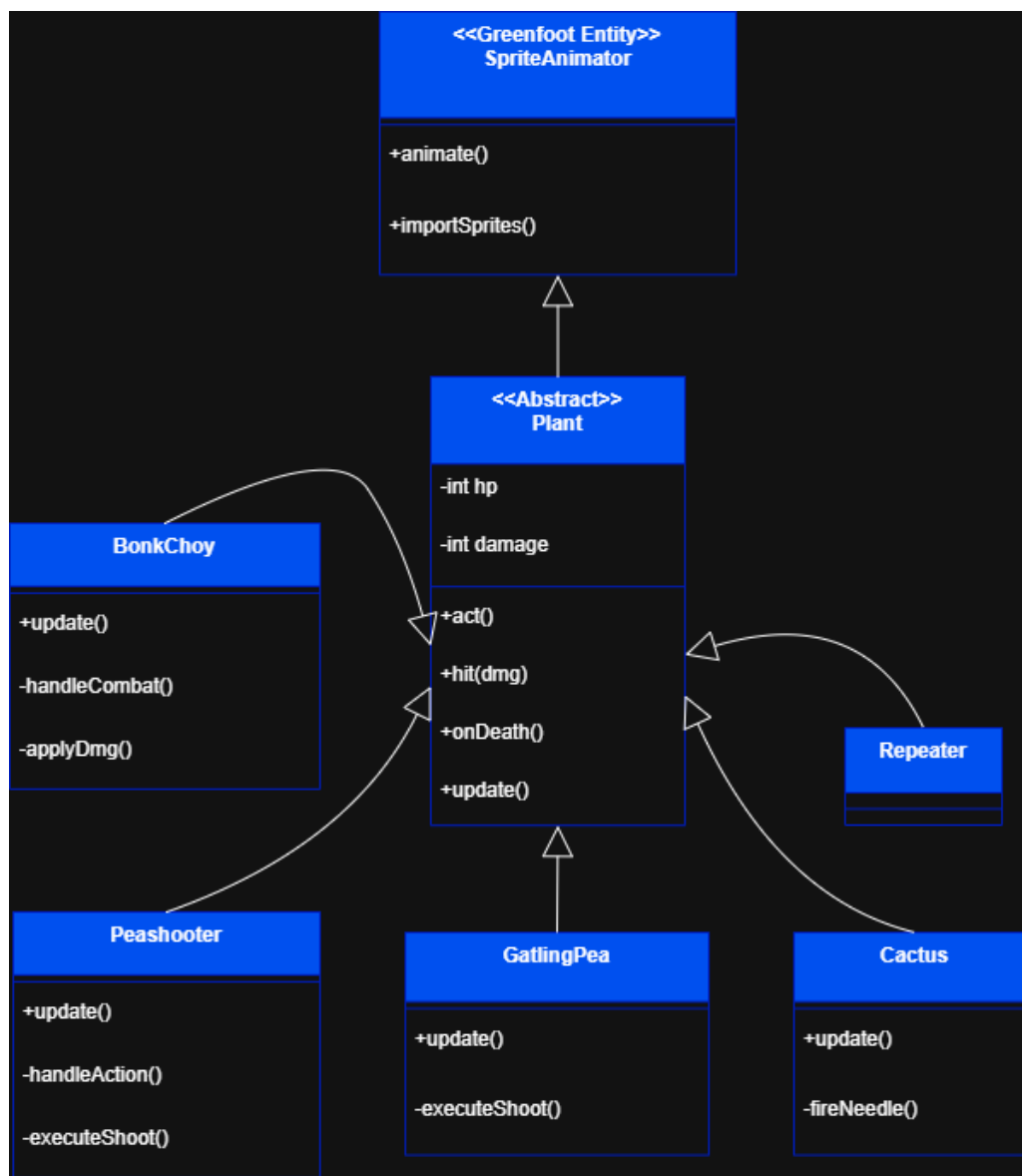


Figure 11: Template Method Pattern model defining invariant algorithmic skeletons for entity systems.

8.5 State Pattern

- **Application:** Manages the dynamic behaviors of in-game objects — including zombies, plants, seed packets, and the core game loop — into distinct, isolated states, completely replacing complex `if-else` branches within the update tick logic.
- **Structural Evidence:**
 - **Zombie System:** The `IZombieState` interface defines lifecycle hooks: `enter()`, `update()`, `exit()`, and `getAnimation()`. The `Zombie` class acts as the *Context*, delegating runtime behaviors to the active `currentState` via `setState()`. Each zombie variant encapsulates its own state machine tailored to its unique characteristics.
 - **Plant System:** The `Plant` class maintains an internal `PlantState` (enum), which is coordinated frame-by-frame by the `PlantStateManager` via `isBusy()` and `canAct()` method evaluations.
 - **SeedPacket System:** The `IPacketState` architecture handles structural state transitions between `AvailableState` and `SelectedState` based on current sun economies and real-time player mouse interactions.
 - **Game Loop System:** The main game loop sequentially transitions through designated global states: `BATTLE` → `WAITING_FOR_REWARD` → `DAVE_TALKING` → `SELECTING_AUGMENT` → `PREPARING_NEXT_WAVE` → `BATTLE`.
- **Benefits:** Every state is encapsulated into its own concrete class or standalone value, making it highly modular to append new states without modifying the original *Context* class, adhering to the **Open/Closed Principle**. Furthermore, state transition logic becomes explicitly defined and far easier to unit-test compared to traditional spaghetti `if-else` structures.

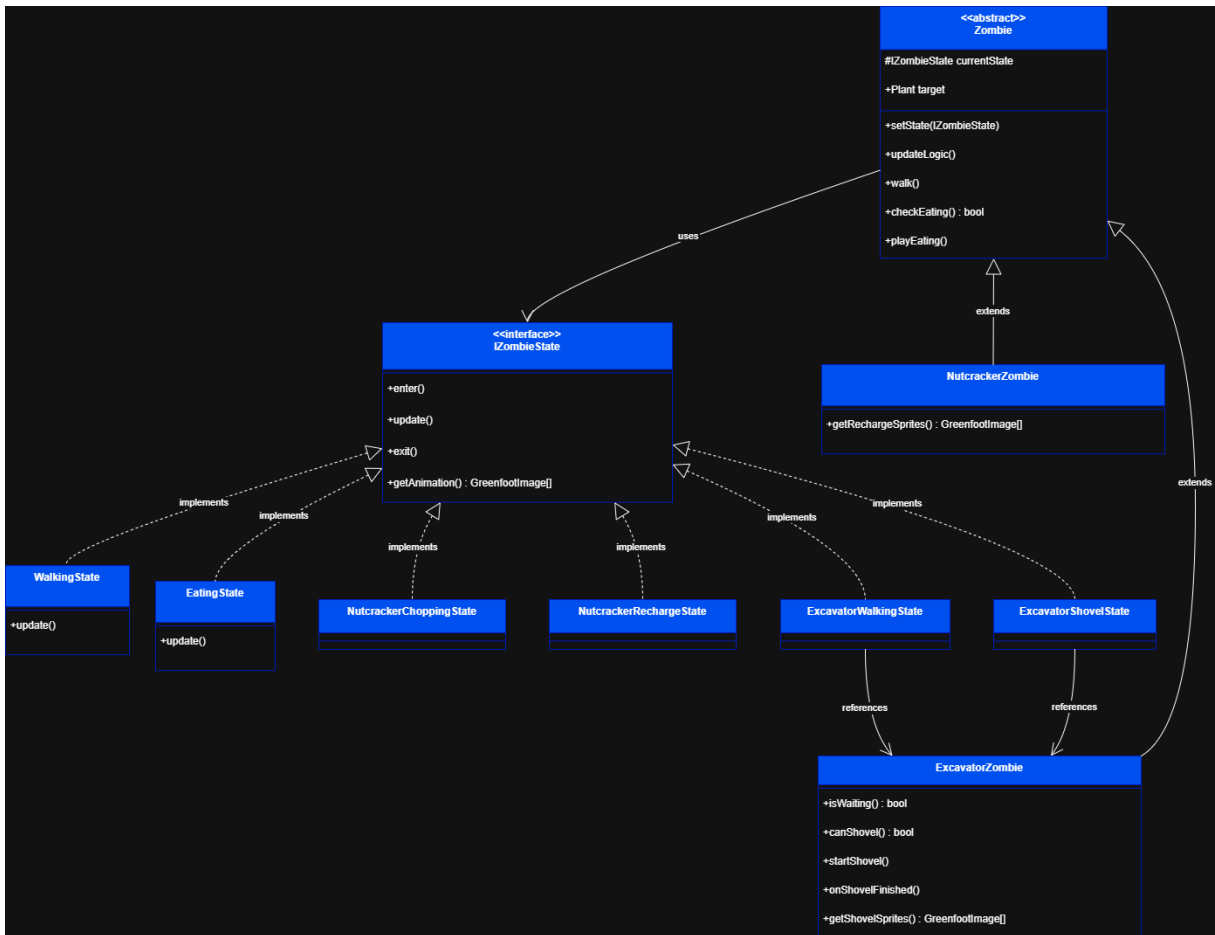


Figure 12: State Pattern applied to the Zombie system: `IZombieState`, `WalkingState`, `EatingState`, and specific behaviors for `NutcrackerZombie` and `ExcavatorZombie`.

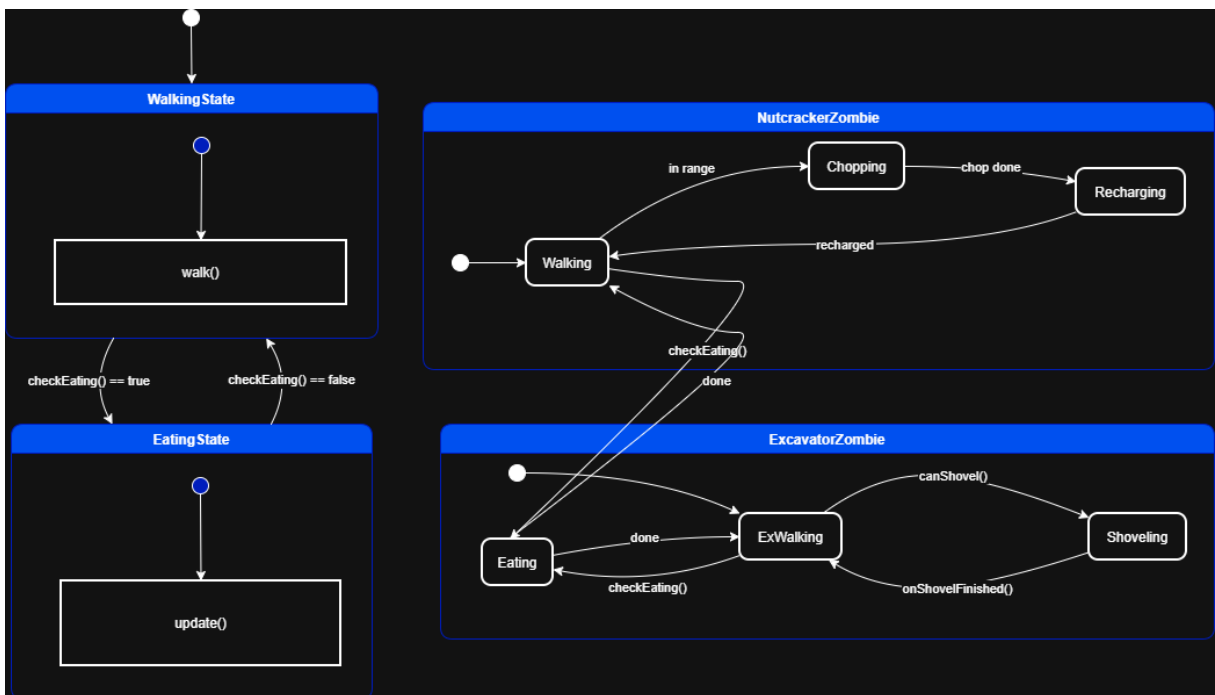


Figure 13: State Pattern applied to the Zombie system: `IZombieState`, `WalkingState`, `EatingState`, and specific behaviors for `NutcrackerZombie` and `ExcavatorZombie`.

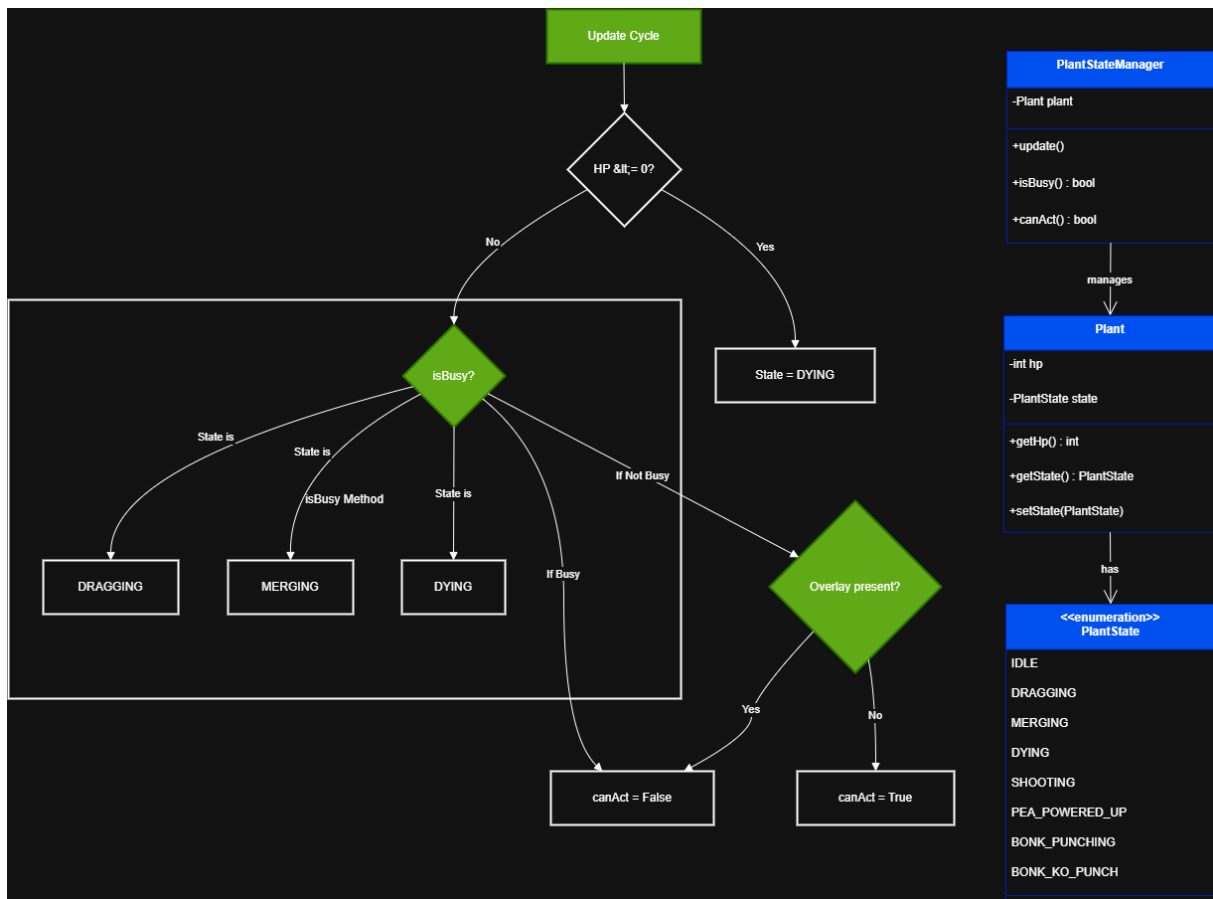


Figure 14: State Pattern applied to the Plant system: `Plant` class, `PlantStateManager`, the `PlantState` enumeration, and `isBusy` execution flow.

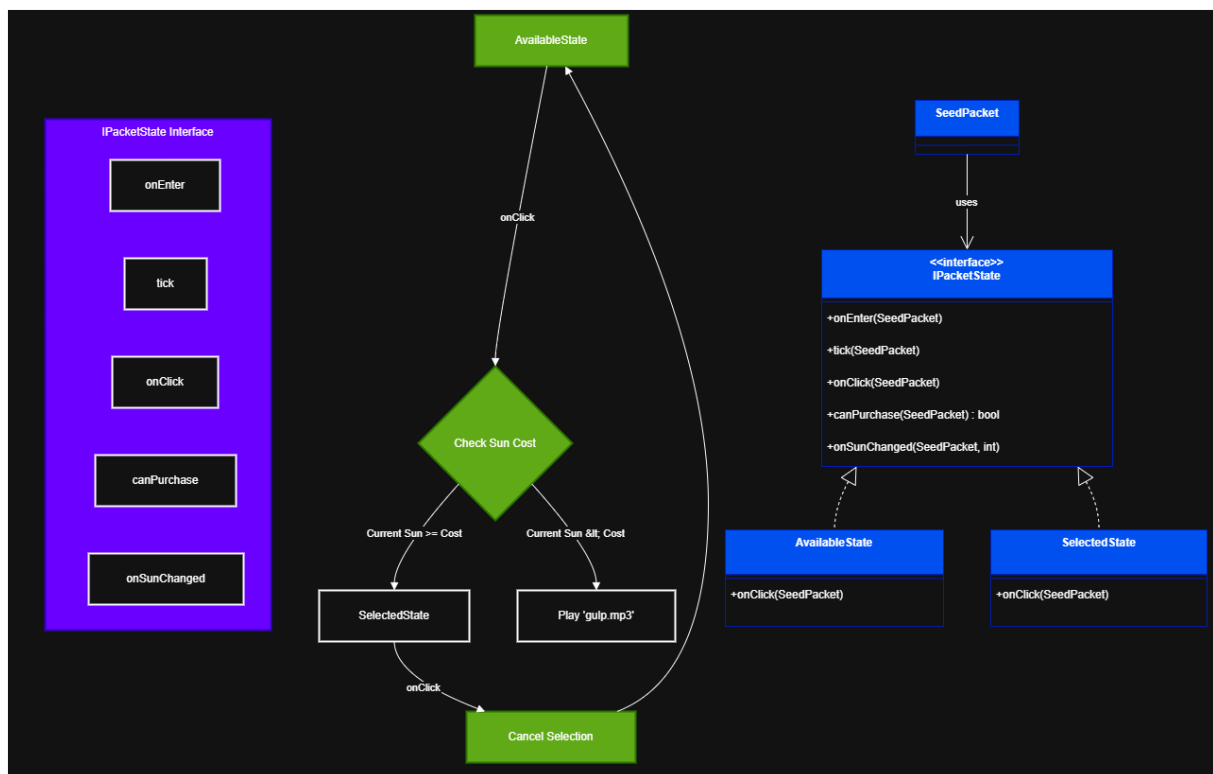


Figure 15: State Pattern applied to the `SeedPacket`: `IPacketState` interface with concrete implementations `AvailableState` and `SelectedState`.

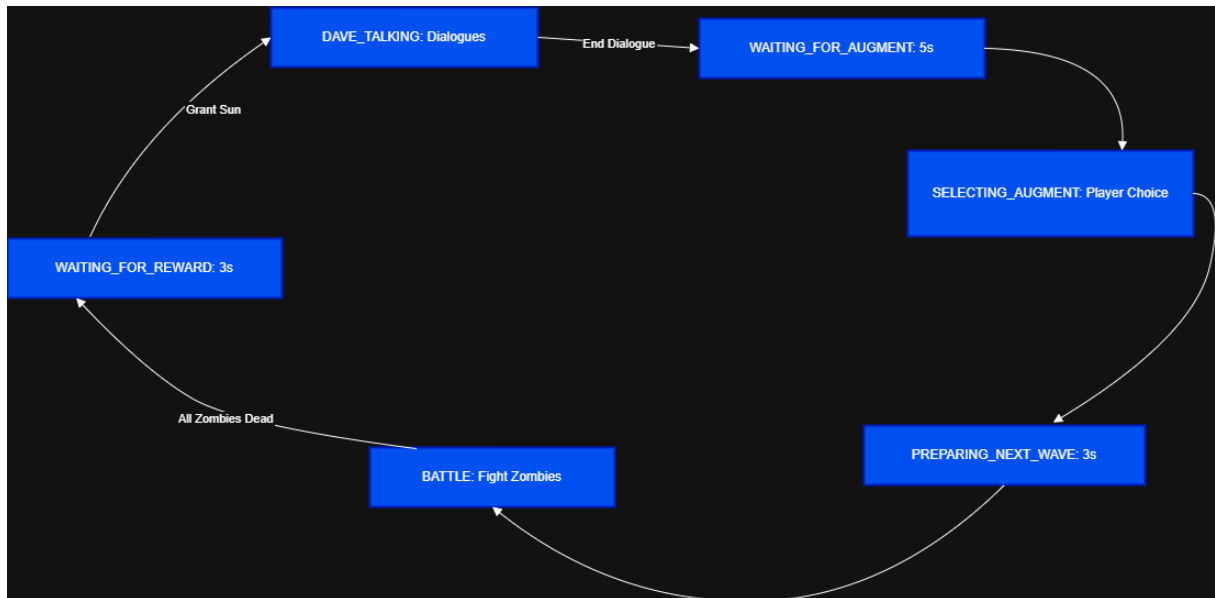


Figure 16: State Pattern applied to the core game loop: state machines for BATTLE, WAITING_FOR_REWARD, DAVE_TALKING, SELECTING_AUGMENT, and PREPARING_NEXT_WAVE.

8.6 Strategy Pattern

8.6.1 Design Intent

The Strategy Pattern is applied to decouple the rarity-based weighted random selection algorithm (*Weighted Random Selection*) from the user interface control class `RollButton`. This mechanism provides the flexibility to alter spawn rates or modify the gacha pool configurations without interfering with the button’s core event-handling logic.

8.6.2 Implementation Components

- **Strategy Context (`RollButton`):** The central class that holds a reference to the strategy collection (`weightedPool`) and triggers the algorithm via the `generateValidPacket()` method.
- **Strategy Implementation (`RarityEntry`):** Encapsulates concrete algorithmic data, including the selection weight (`weight`) and its corresponding card entity class (`packetClass`).
- **Strategy Product (`SeedPacket`):** The resulting object instantiated after the execution of the algorithm (e.g., `PeashooterPacket`, `SunflowerPacket`).

8.6.3 Concrete Evidence

When a player triggers a roll, the `generateValidPacket(pool, totalWeight)` method inside `RollButton` iterates through the list of input `RarityEntry` elements. Instead of using rigid `if-else` branching statements to evaluate each plant variant, the system utilizes a *Reflection* mechanism based on the meta-information of the winning `RarityEntry` to automatically instantiate the respective `SeedPacket` object.

8.6.4 Benefits Achieved

- **Encapsulation:** Completely isolates the gacha rate calculation logic within the `weightedPool` structure, keeping the `RollButton` UI class source code clean and explicit.
- **Flexibility:** Effortlessly updates, expands, or re-balances the winning probabilities of new seed variants by simply adding a new `RarityEntry` element, without affecting the core codebase.

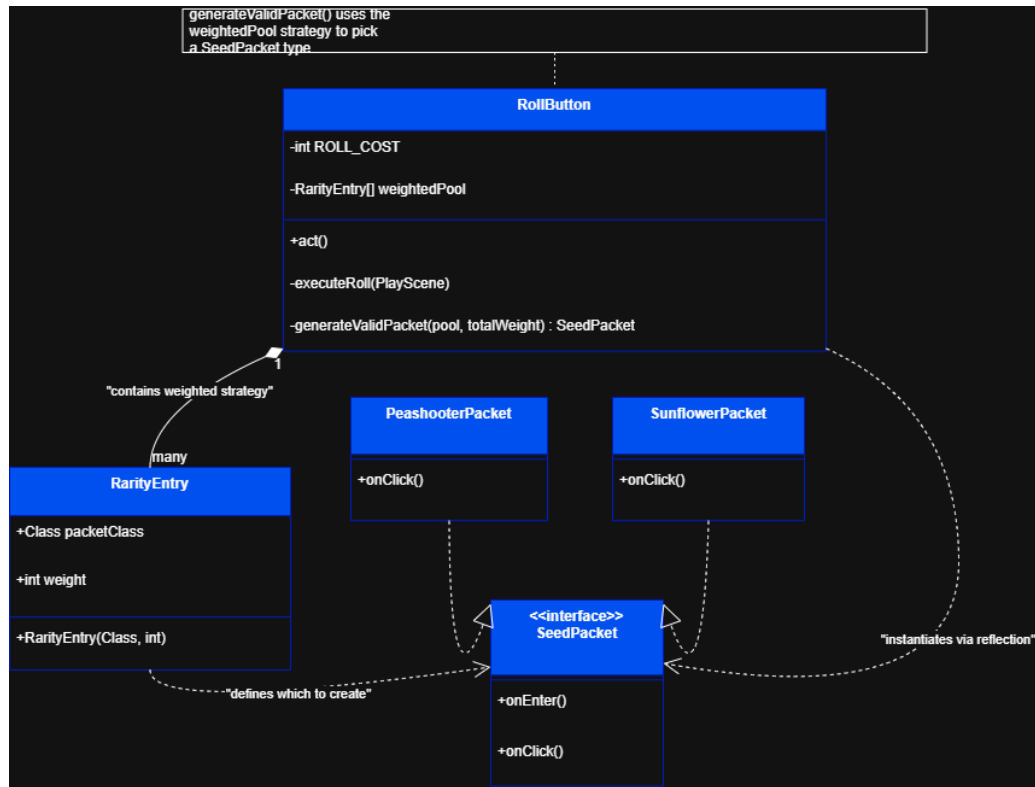


Figure 17: Strategy Pattern model applied in the weighted card-rolling mechanism.

8.7 Command Pattern

8.7.1 Design Intent

The Command Pattern is implemented to cleanly encapsulate the multi-step plant-merging mechanism—including frame-by-frame visual interpolation, multi-entity arrival synchronization, and state mutations—into independent object representations. This architectural choice decouples complex asynchronous game logic from the primary engine loop of `PlayScene`, facilitating seamless extensibility and providing native support for cascading state rollbacks (*undo operations*).

8.7.2 Implementation Components & Evidence

The architecture meticulously follows the Gang of Four (GoF) structural specifications:

- **Command Interface (`PlantCommand`):** Defines the behavioral contract via abstract `execute()` and `undo()` operations, unifying all merge-related transactions.
- **Concrete Commands (`MergeCommand` & `MergeAnimationCommand`):**
 - `MergeAnimationCommand` governs the real-time visual glide of source plants toward the target cell via a frame-based `tick()` cycle.
 - `MergeCommand` orchestrates the core business logic, handling arrival synchronization via `notifyArrived()` and replacing base units with higher-tier entities.
- **Invoker (`PlantCommandInvoker`):** Maintains a sequential execution history within a `Stack<PlantCommand>`. It triggers operations natively and hosts the `undoLast()` routine to reverse merge sequences without exposing domain logic.
- **Client/Utility (`PlantCombineHandler`):** Scans the grid, evaluates match-three upgrade criteria, injects spatial contexts, and instantiates the concrete commands into the execution engine.
- **Receiver (`Plant`):** The ultimate domain target acting on spatial re-allocations and state shifts (`setState()`) ordered by the active commands.

8.7.3 Benefits Achieved

- **Decoupling of Animation and Logic:** Isolates volatile rendering dependencies from the foundational game state, ensuring that mechanical execution depends strictly on verified spatial arrival rather than hardcoded timers.
- **Reversibility & Robust Error-Handling:** The inclusion of explicit `undo()` subroutines enables immediate command cancellation or historical recovery if a participating entity is destroyed midway by enemy interactions.

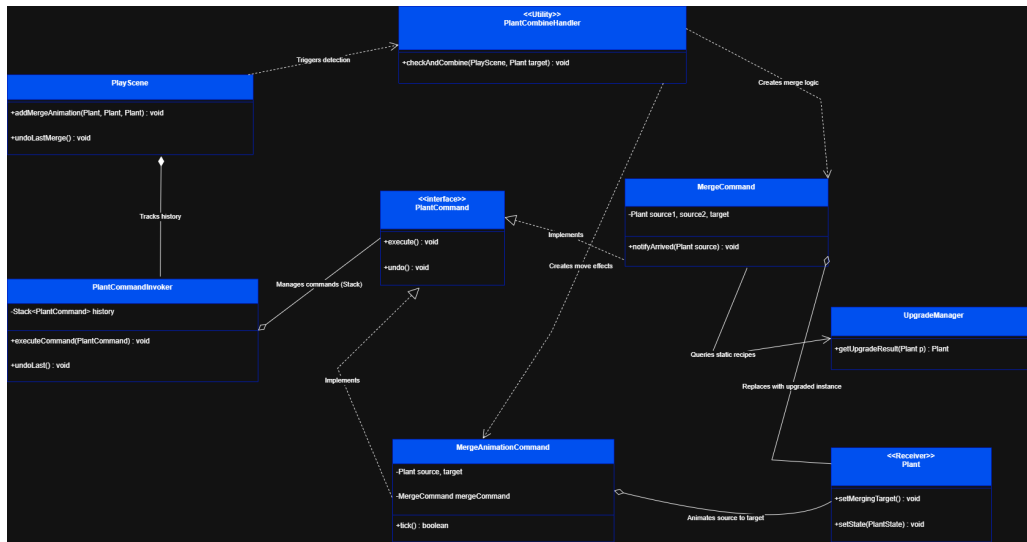


Figure 18: UML Class Diagram of the Command Pattern implementation managing plant upgrade workflows.

8.8 Observer Pattern (Event Bus)

8.8.1 Design Intent

The Observer Pattern is implemented as an intermediary **Event Bus** to dispatch lifecycle events (such as taking damage, death, and merging) to multiple peripheral modules (UI, Audio, Grid). This mechanism eliminates direct dependencies from game entities (**Plant**, **Zombie**) on high-level systems, optimizing encapsulation.

8.8.2 Implementation Components & Evidence

The system is divided into two independent channels (**PlantEventBus** and **ZombieEventBus**) featuring the following components:

- **Subject / Publisher:** The signaling entity. When a **Zombie** dies, it invokes `eventBus.publishDeath(this)` without needing to know which sub-systems will handle the aftermath.
- **Event Bus:** The central coordinator, maintaining active subscription lists and handling event forwarding (*dispatching*) to registered components.
- **Subscriber Interfaces** (**IPlantEventListener**, **IZombieEventListener**): Defines standard callback structures (*callback*). Modules like **GridManager** implement these interfaces to listen for `onPlantMerged()` events, automatically clearing or updating grid cell occupations.

8.8.3 Benefits Achieved

- **Single Responsibility (SRP):** Completely isolates game entities from peripheral side-effects; plants and zombies focus exclusively on pure survival and combat logic.
- **Extensibility:** Enables seamless integration of new features (such as an Achievement System) by simply registering a new Subscriber, without requiring any modifications or recompilation of the core entity source code.



Figure 19: Event Bus architecture decoupling the connection between the Subject and system Subscribers.

9 Conclusion and Future Work

The **Plants vs Zombies: Golden Spatula (Greenfoot)** project has successfully demonstrated the practical application of core **Object-Oriented Programming (OOP)** principles in game development. By leveraging the Greenfoot framework, the project focuses on object lifecycle management and intuitive interactions between entities through a strict class hierarchy.

Results Achieved:

- **Core Gameplay:** Successfully developed a custom **World** featuring the fundamental gameplay loop of PvZ on a hex-grid coordinate system.
- **Pure OOP Architecture:** Extensively applied *Inheritance* and *Polymorphism*. Entities such as plants and zombies are logically organized under base classes (**Plant**, **Zombie**), optimizing code reusability.
- **Game Mechanics:** Effectively implemented the sun-based economy system, plant placement mechanics, and zombie wave spawning management.
- **System Integrity:** Utilized Greenfoot's built-in tools to construct an accurate collision detection system and an efficient object cleanup mechanism.
- **Design Standards:** Strictly adhered to **SOLID** principles, establishing a clean, maintainable, and scalable project structure.

10 Links

SOURCES:

- [1] <https://github.com/rohangoel96/PlantsVsZombies-Game>
- [2] <https://github.com/GeeeeekExplorer/PVZ>
- [3] https://github.com/Huy-IU-2k6/Plant_vs_Zombie
- [4] <https://github.com/arminkz/PlantsVsZombies>
- [5] <https://github.com/BhavyaC16/Plants-Vs-Zombies>
- [6] <https://github.com/AndreAylas/202223-PlantsVsZombies>
- [7] <https://github.com/marcomarrelli/OOP23-pvz-clone>
- [8] <https://github.com/dominhduy09/university-documents/tree/main/OOP%20-%20Lab/Lam%20Game>

OUR GITHUB CODES:

- [9] <https://github.com/KietAnhCS/Plants-vs-Zombies>